Assignment 1: Multi-Threaded Online Food Delivery System (Zomato/Swiggy Simulation)

🎯 Objective

Design a **multi-threaded food delivery backend simulation** where multiple orders are processed concurrently while maintaining data consistency.

📌 Problem Statement

You are building a backend system for a food delivery platform. Multiple customers place orders simultaneously, and the system must:

- Accept orders concurrently
- Assign delivery agents
- Process payments
- Update shared restaurant inventory safely

System Requirements

1. Order Processing (Thread Creation)

- Each order should be processed using:
  - **1 class extending Thread**
  - **1 class implementing Runnable**
- At least **10 orders must be processed simultaneously**

---

2. Thread Priority

- Assign priorities based on order type:
  - VIP Order → MAX_PRIORITY
  - Regular Order → NORM_PRIORITY
  - Low priority/test order → MIN_PRIORITY

---

3. Multiple Thread Execution

- Simulate:
  - 10–15 concurrent customer orders
- Each thread should:
  - Print order ID
  - Processing time
  - Assigned delivery agent

---

4. Synchronization (Critical Section)

- Shared resource: **Restaurant Inventory**
- Example:
  - Only 5 pizzas available
  - Multiple threads try to order pizza

☞ Use:

- `synchronized` method/block OR
- `ReentrantLock`

Ensure:

- No over-selling occurs
- Inventory updates are thread-safe

---

5. Executor Service (Thread Pool)

- Instead of creating threads manually for all orders:
  - Use `ExecutorService` with a fixed thread pool of size 3 or 5

☞ Requirement:

- Submit all order tasks to executor
- Shutdown executor gracefully

---

6. Daemon Thread

- Create a **background logging service**:
  - Logs system activity every 3 seconds
  - Should run as a **daemon thread**

Input Constraints

- Total Orders: 10–15
- Inventory:
  - Pizza = 5
  - Burger = 10

Expected Output (Sample)

```
[Order-1] VIP Order received
[Order-2] Regular Order received
[Order-3] Low Priority Order received

Processing Order-1 by Thread-1
Inventory updated: Pizza remaining = 4

Processing Order-2 by Thread-2
Inventory updated: Pizza remaining = 3

Logger: System running smoothly...

Order-5 failed: Pizza out of stock
```

📁 Deliverables

1. Source Code (well-structured packages)
2. Output Screenshot
3. Explanation Document:
   - Thread creation approach used
   - Where synchronization is applied
   - Why ExecutorService is used
   - Role of daemon thread

Assignment 2: Banking System with Concurrent Transactions

🎯 Objective

Simulate a **multi-threaded banking system** handling concurrent deposits, withdrawals, and balance checks with proper synchronization and thread management.

Problem Statement

Design a banking system where multiple users perform transactions simultaneously on shared bank accounts.

System Requirements

1. Thread Creation

- Create transaction threads using:
  - One class extending `Thread`
  - One class implementing `Runnable`

---

2. Multiple Threads Execution

- Simulate:
  - 5 users performing transactions on the same account
- Each user performs:
  - Deposit
  - Withdrawal
  - Balance check

---

3. Synchronization (Critical Section)

- Shared Resource: **Bank Account Balance**

☞ Requirements:

- Prevent:
  - Race conditions
  - Negative balance
- Use:
  - `synchronized` OR
  - `Lock`

---

4. Thread Priority

- Assign priorities:
  - Fraud detection thread → MAX_PRIORITY
  - Normal user → NORM_PRIORITY
  - Background reporting → MIN_PRIORITY

---

5. Executor Service

- Use `ExecutorService`:
  - Fixed thread pool of 3
- Submit all transaction tasks

---

6. Daemon Thread

- Create a **Fraud Detection Service**:
  - Runs continuously
  - Monitors suspicious activity
  - Runs as daemon thread

Input Data

Initial Balance: ₹10,000

Transactions:

- User 1: Withdraw ₹5000
- User 2: Withdraw ₹7000

- User 3: Deposit ₹3000
- User 4: Withdraw ₹2000
- User 5: Deposit ₹1000

Expected Output (Sample)

```
User-1 withdrawing ₹5000
Balance after withdrawal: ₹5000

User-2 withdrawing ₹7000
Transaction Failed: Insufficient Balance

User-3 depositing ₹3000
Balance updated: ₹8000

Fraud Detection Active...
```

📁 Deliverables

1. Java Source Code
2. Console Output
3. Short Report:
    - How race condition was prevented
    - Difference between Thread vs Runnable used
    - Why ExecutorService is better than manual threads
    - Role of daemon thread